END-TO-END SOC LAB REPORT

SQL INJECTION DETECTION & ANALYSIS USING SPLUNK SIEM

Author: Imafidon Eromosele Joseph

Program: Cybersecurity

Project Type: Security Operations Center (SOC) Lab

Date: 03/04/2026

Abstract

This report presents the design and implementation of a SOC lab focused on detecting SQL injection attacks using Splunk SIEM. It demonstrates log analysis, real-time alerting, and incident investigation in a controlled environment.

Tools & Technologies Used

- Kali Linux
- DVWA
- Splunk Enterprise
- Apache2
- MySQL
- Gmail SMTP

Introduction

Web applications are common targets for cyberattacks, with SQL Injection being one of the most critical vulnerabilities. This project aims to simulate such attacks and build a detection mechanism using a SIEM platform.

The objective is to demonstrate how SOC analysts: Monitor logs, Detect malicious patterns, Respond to security incidents.

Objectives

- Simulate SQL Injection attacks in a controlled lab
- Collect and analyze web server logs
- Develop detection queries using Splunk SPL
- Configure real-time alerts
- Visualize attacks using dashboards

System Architecture

The lab environment consists of:

- Attacker Machine: Kali Linux
- Target Application: DVWA (Damn Vulnerable Web Application)
- SIEM Tool: Splunk Enterprise
- Log Source: Web server logs (Apache2)
- Alert Mechanism: Email (SMTP via Gmail)

Methodology

Environment Setup

- Installed Splunk Enterprise on Kali Linux
- Configured DVWA for vulnerability testing
- Enabled logging on the web server

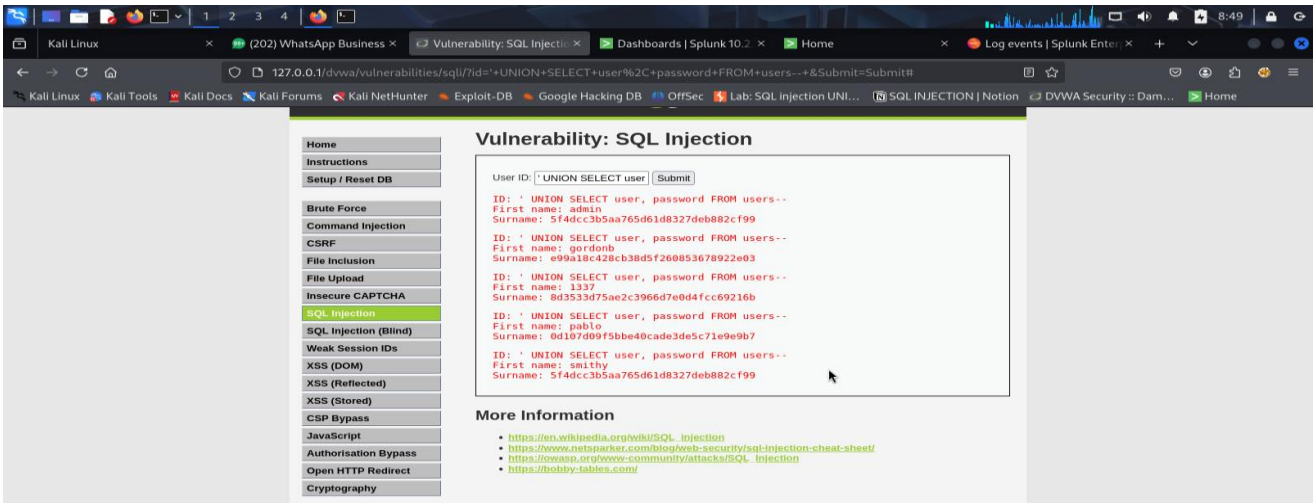
Attack Simulation

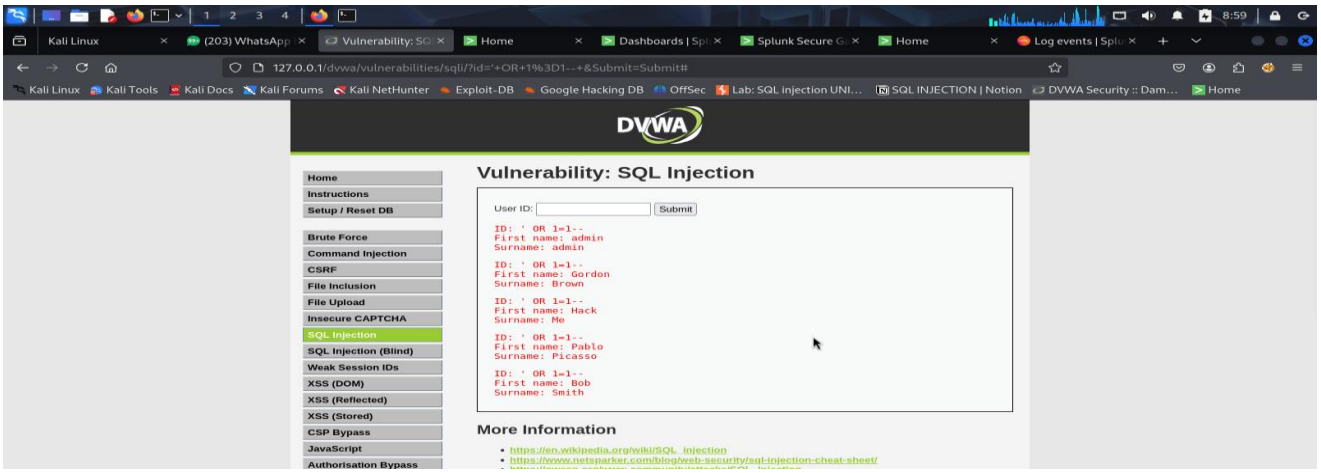
SQL Injection payloads used include:

' OR 1=1--

' UNION SELECT user, password FROM users--

' ORDER BY 1--



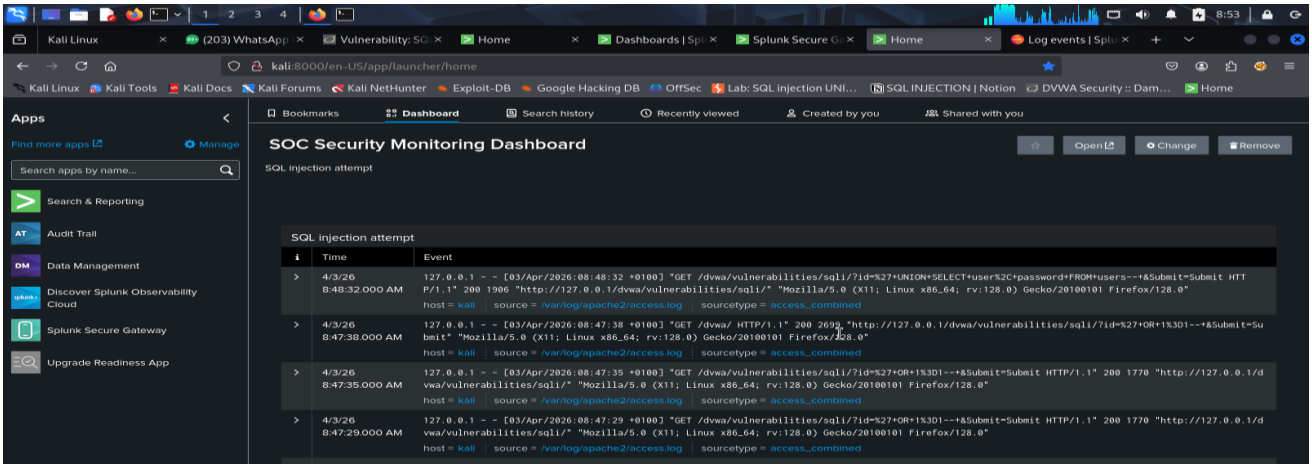


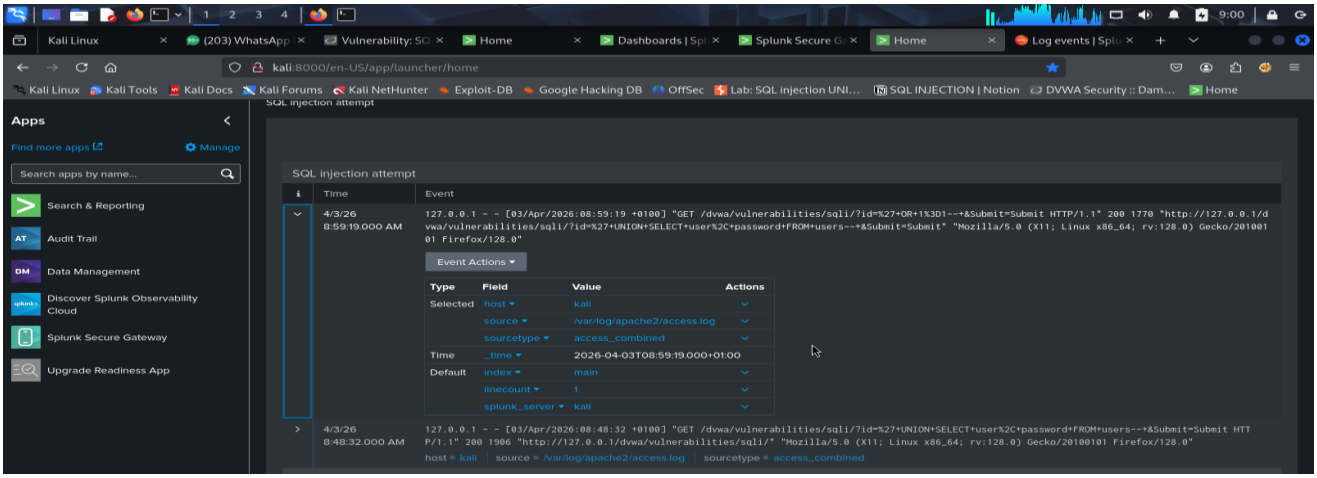
These payloads were used to:

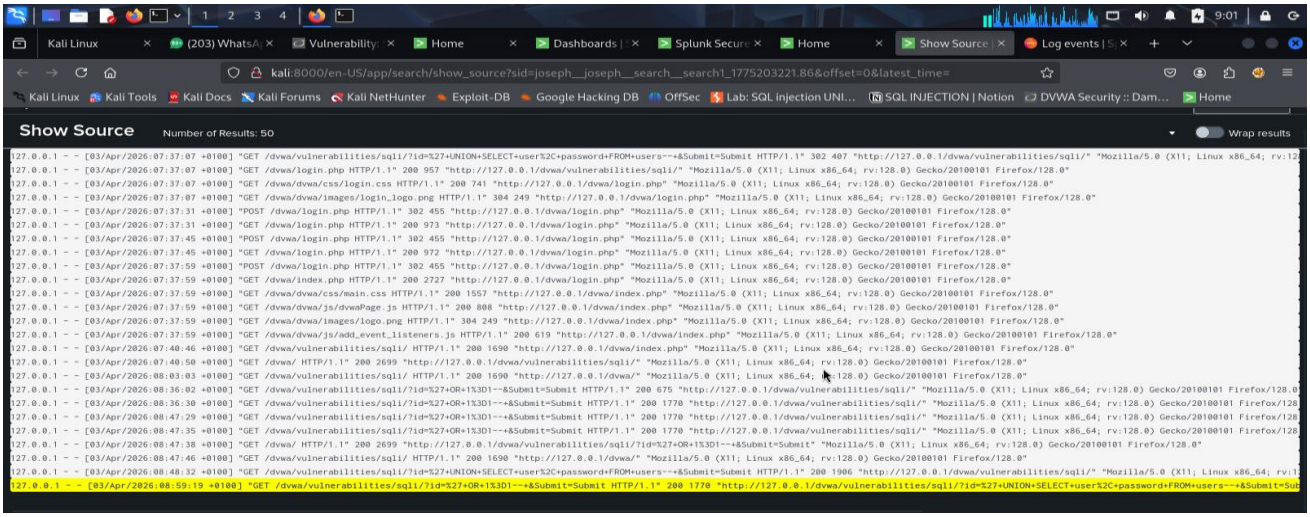
- Bypass authentication
- Extract database information
- Test input validation weaknesses

Log Collection

- HTTP requests were captured in web logs
- Logs were ingested into Splunk
- Fields extracted:
- client IP
- request URI
- query strings

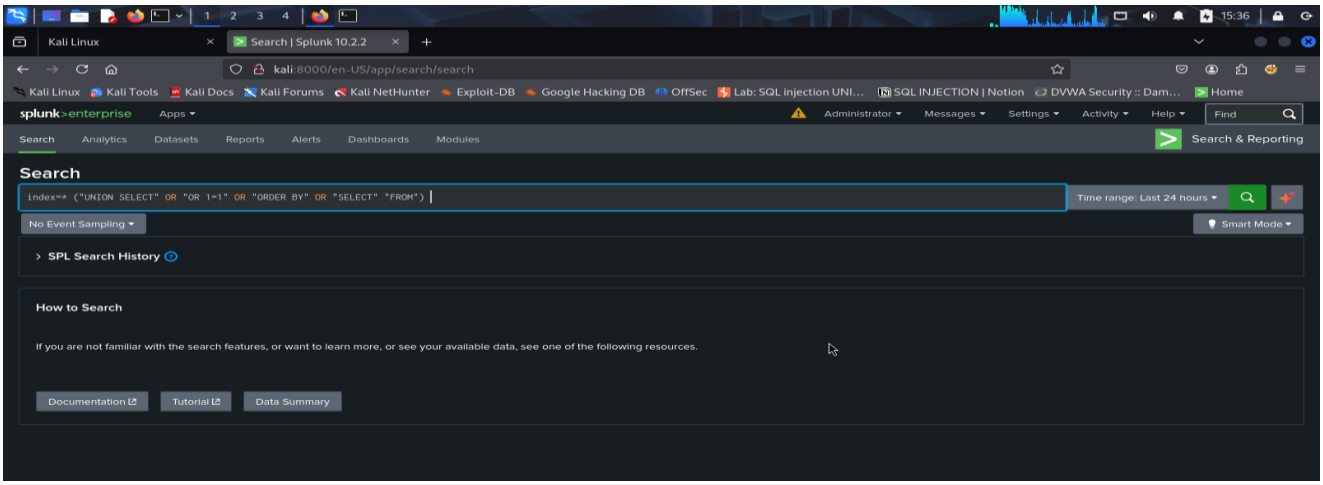


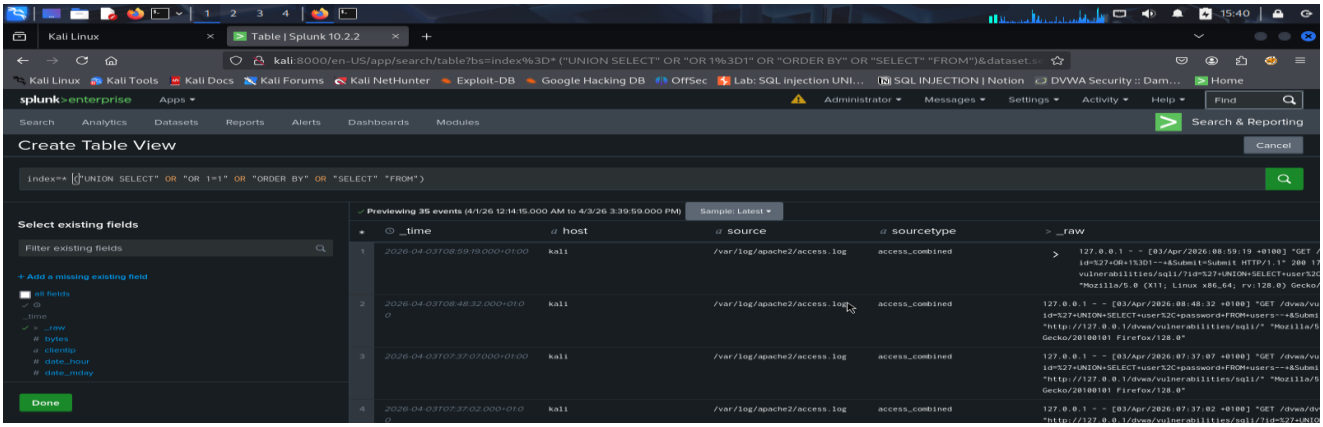




Detection Rule (SPL Query)

index=web_logs ("UNION SELECT" OR "OR 1=1" OR "ORDER BY")

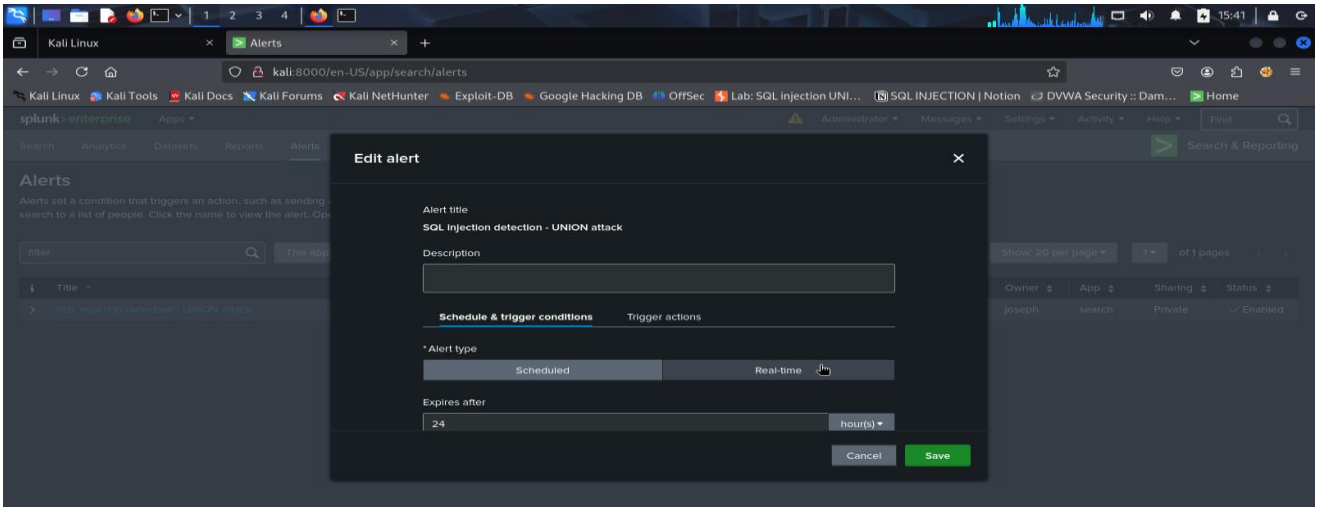


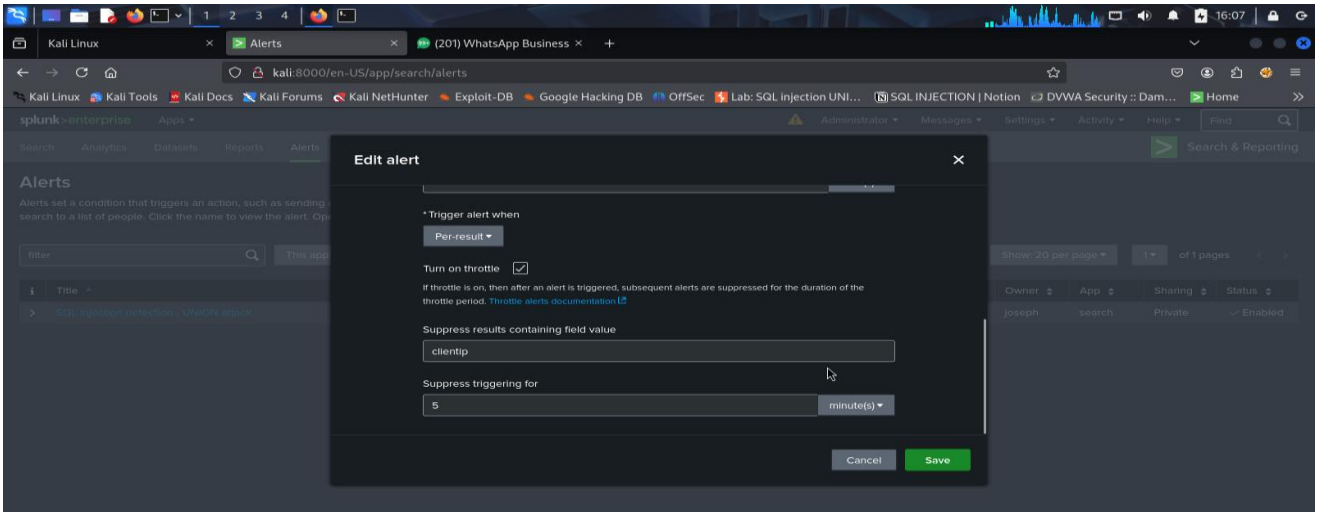


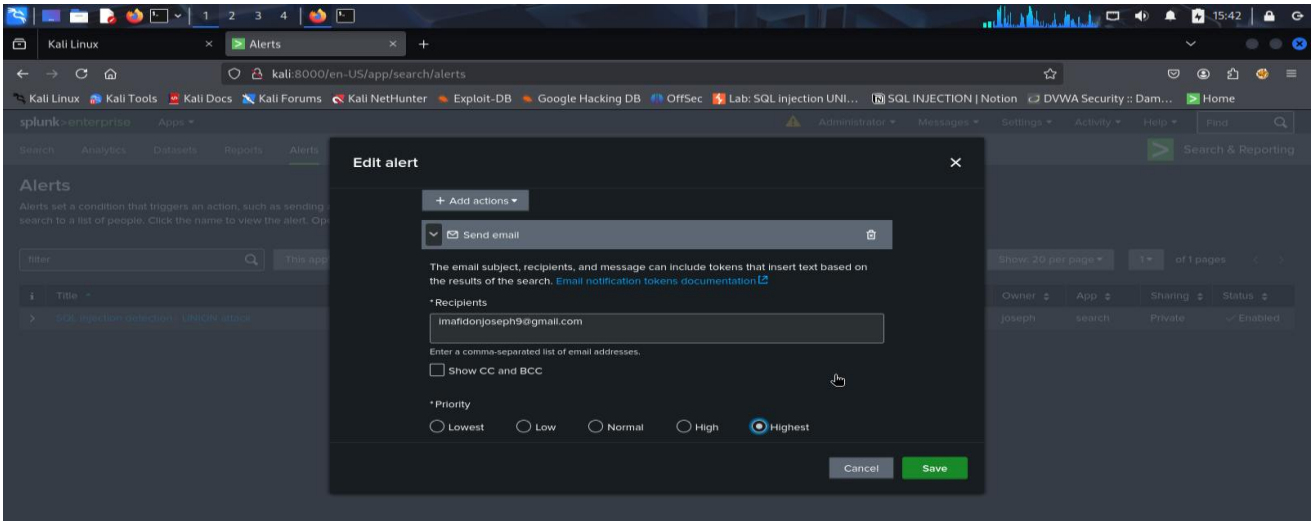
This query identifies suspicious SQL patterns and groups them by source IP, and give detail's like attack time, date, attacker host, and SQL commands used.

Alert Configuration

- Alert Type: Real-time
- Trigger: Per-result
- Action: Email notification
- Throttling: Optional (to reduce alert flooding)







Results and Findings

- SQL Injection attempts were successfully detected
- Alerts were triggered in real time
- Email notifications were configured
- Dashboard provided clear visualization of attack trends

Analysis

The detection mechanism effectively identified SQL Injection attempts based on known patterns. However, reliance on keyword-based detection may lead to false positives or missed obfuscated attacks.

This highlights the need for:

- Advanced detection techniques
- Behavioral analysis
- Continuous tuning of SIEM rules

Security Recommendations

- Implement input validation and sanitization
- Use prepared statements in database queries
- Deploy Web Application Firewalls (WAF)
- Enable continuous monitoring and logging
- Apply least privilege principles

Future Improvements

- Integrate machine learning-based anomaly detection
- Automate responses using SOAR tools
- Expand detection to other attack types (XSS, RCE)
- Deploy lab in cloud environment for remote access

Conclusion

This project successfully demonstrates an SOC workflow, from attack simulation to detection. It highlights the importance of SIEM tools in identifying and mitigating web-based attacks and provides practical experience relevant to real-world cybersecurity operations.